Recursive Self-Improvement for LLM Agents

Google

Harness^2: Recursive Agent Harnessing for an Open World

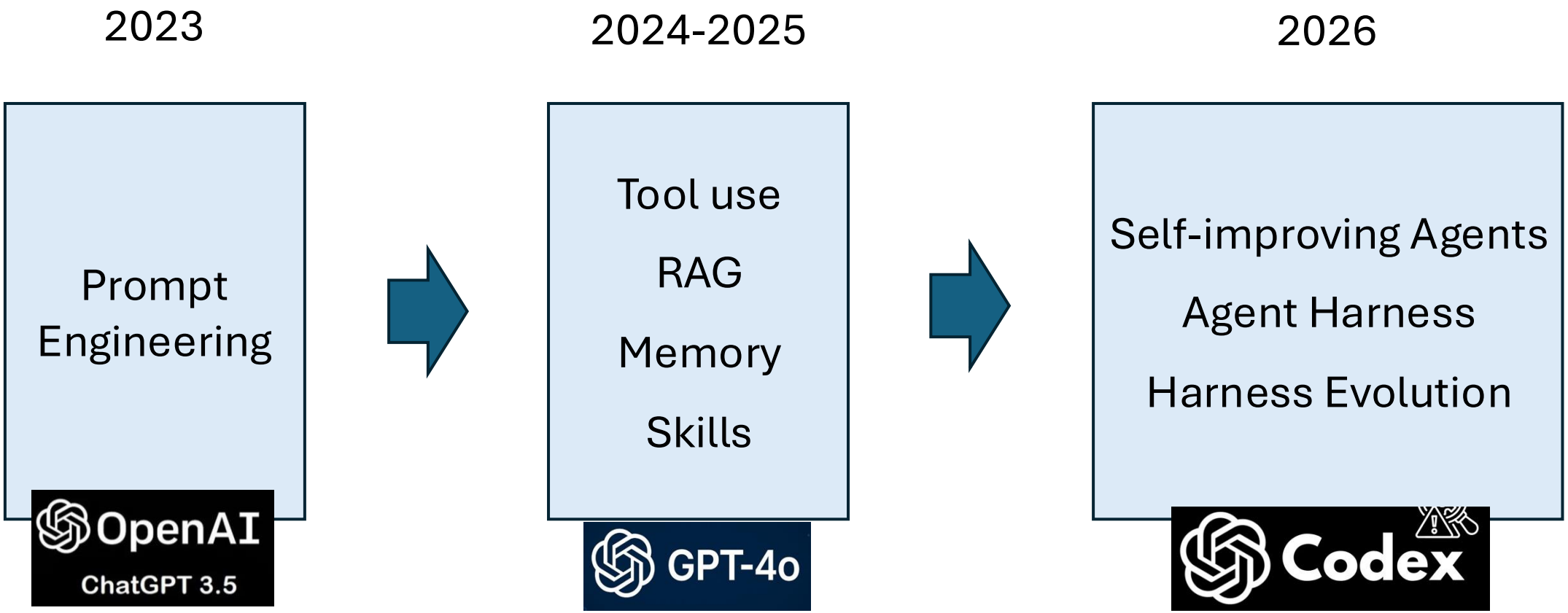
5th Nov

Shuai Wang

https://wangshuaiia.github.io/reading-club/

Evolution of LLM-based Agents

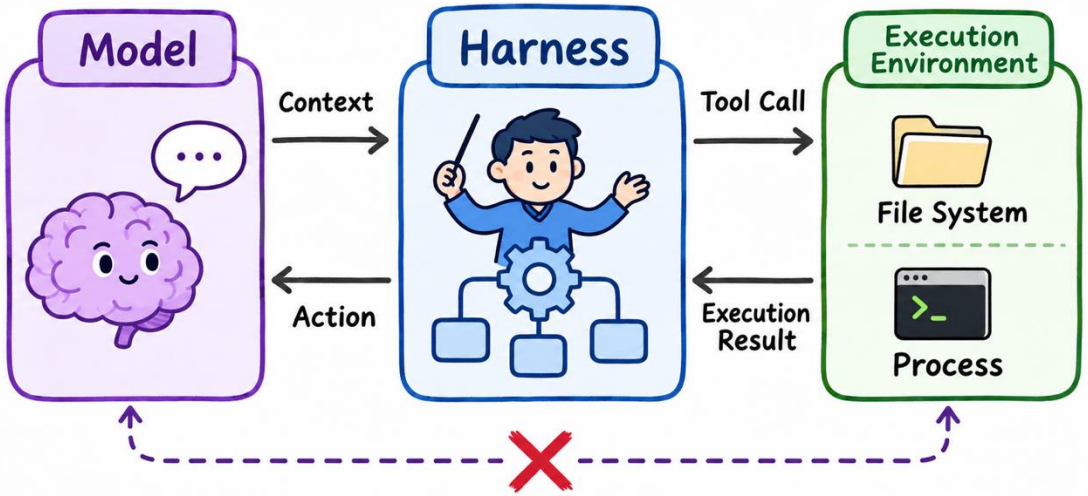
- From prompting to augmentation to self-improvement.



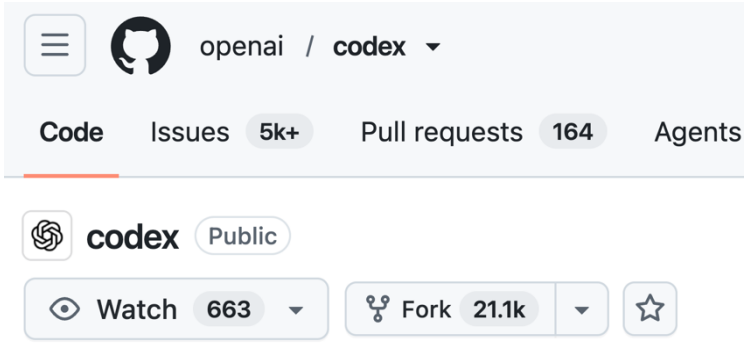
Harness

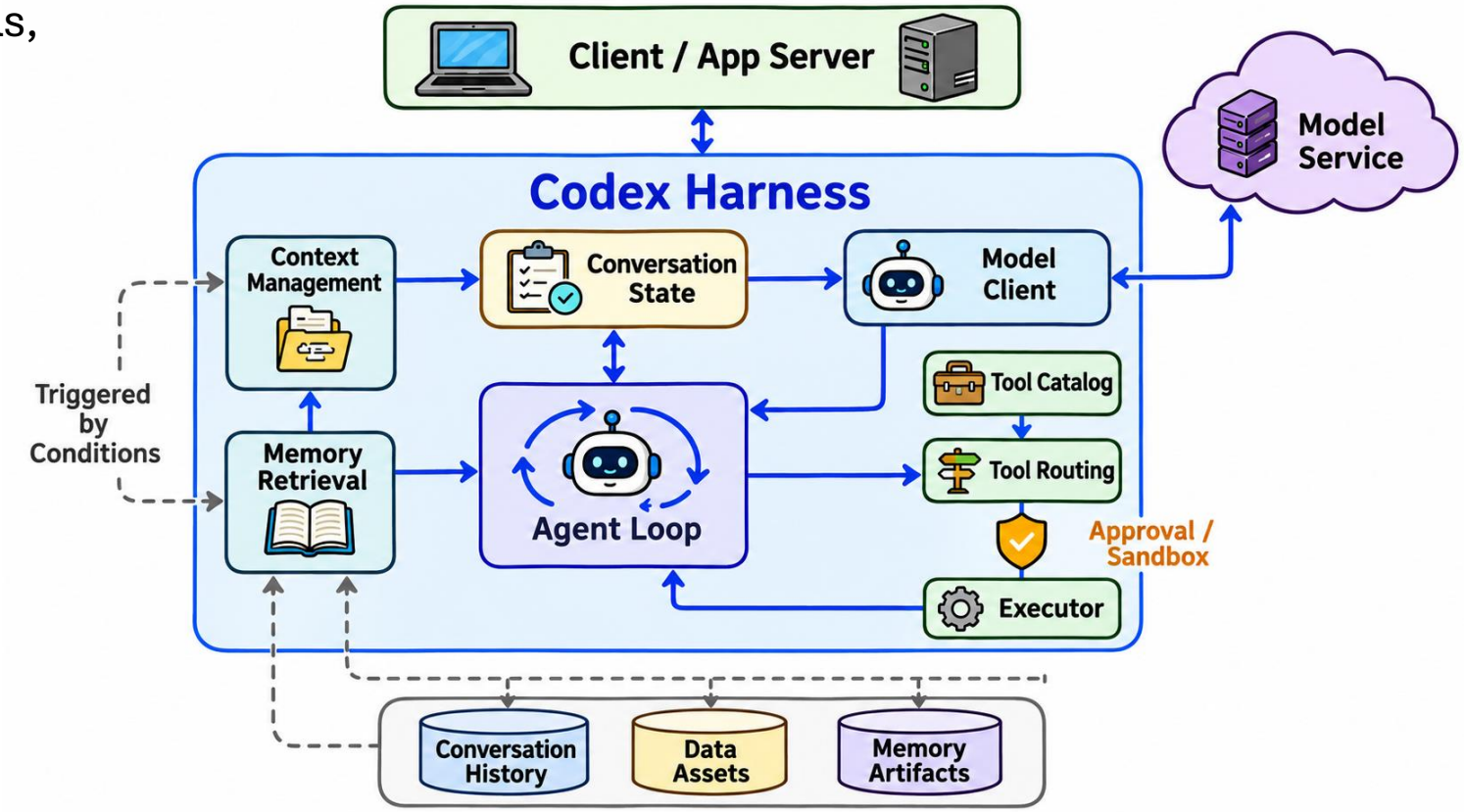
➢ What is a Harness?

- The harness is the surrounding system through which the LLMs performs tasks.
- It includes prompts, memory, skills, tools, and orchestration.



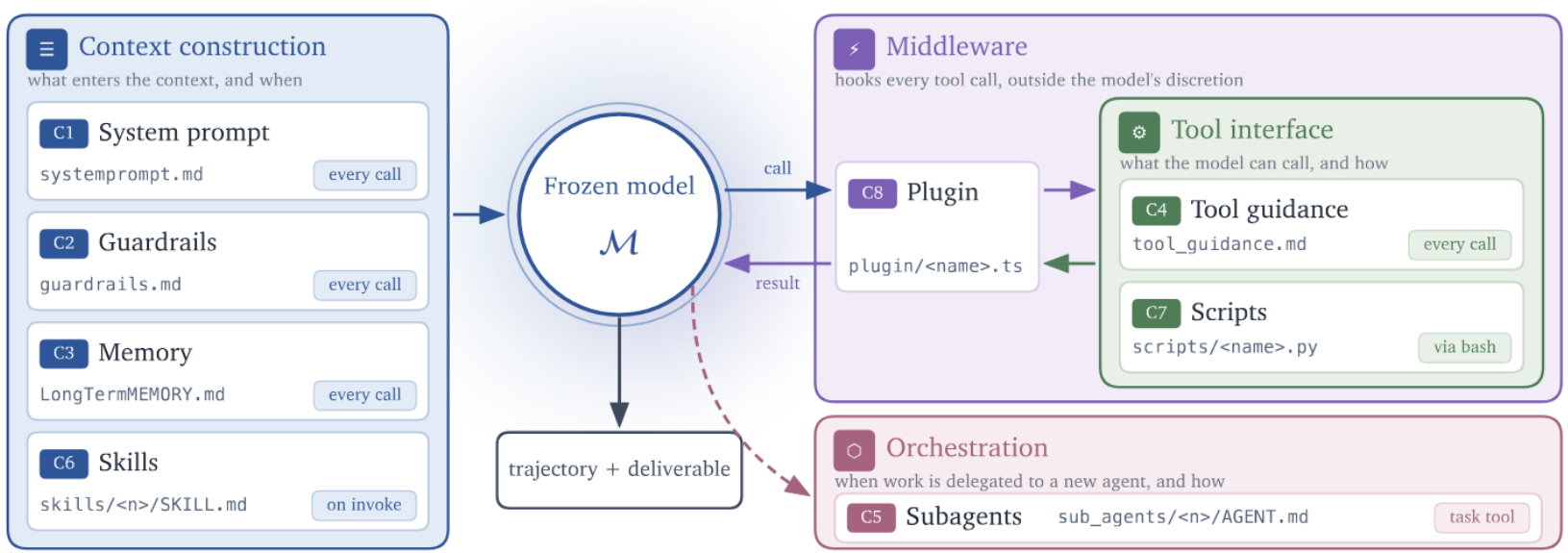
➢ Codex Harness



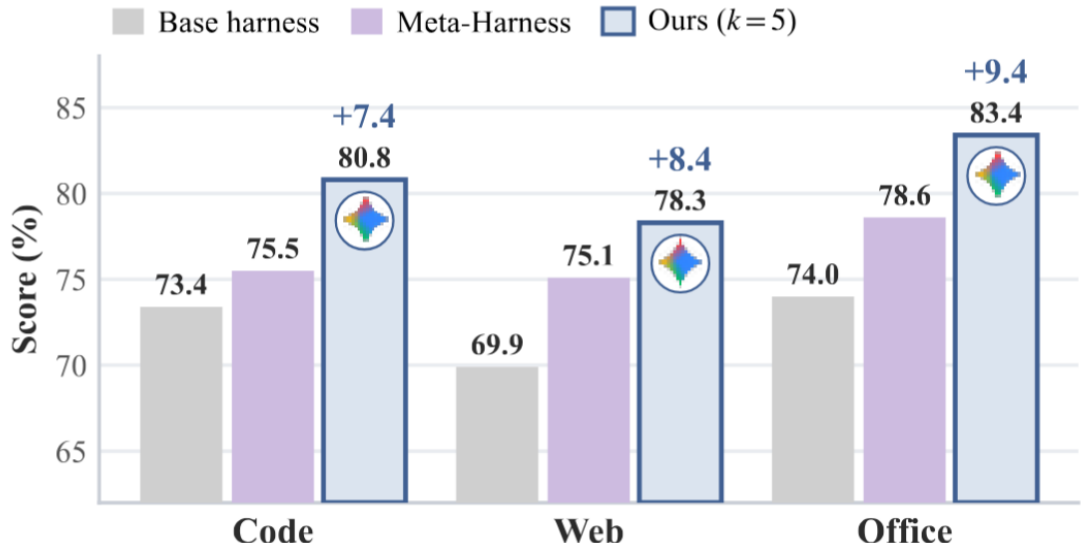


Why Harness Important

- **The eight harness components in Harness²**



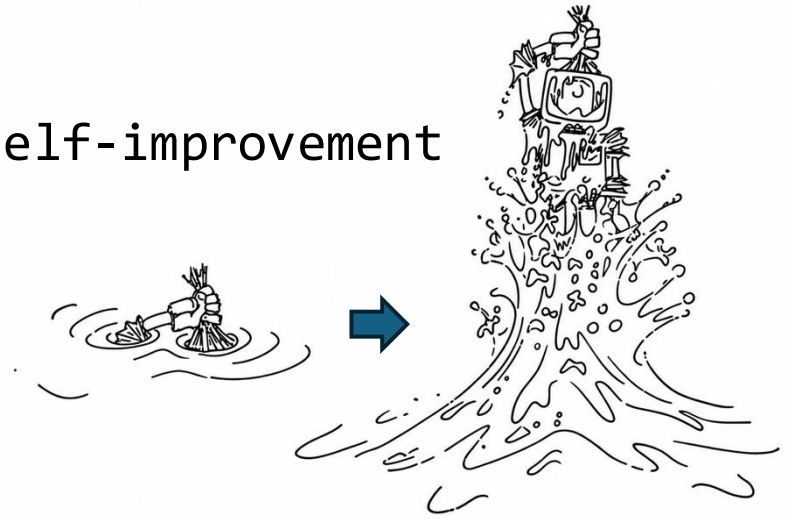
- **Why does it matter?**
  - Same LLM, different harness: very different performance.



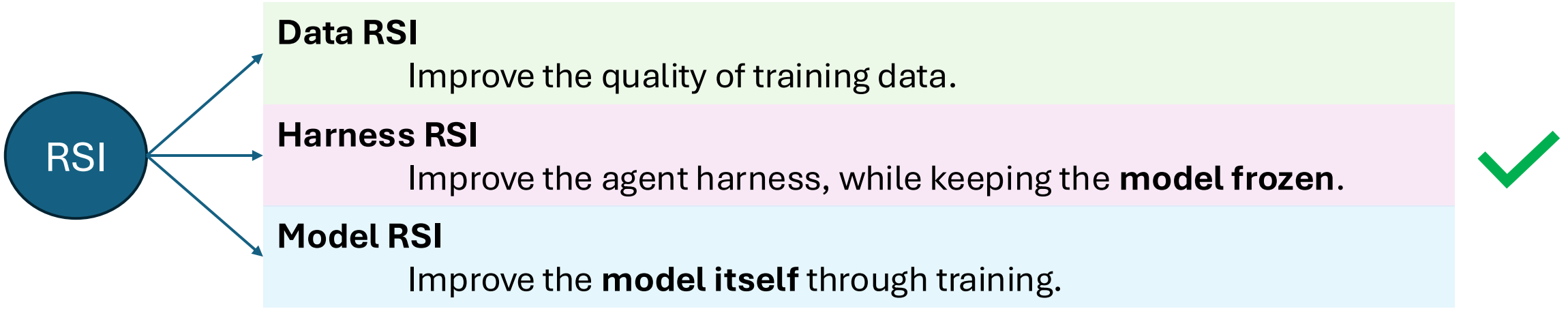
Recursive Self-Improvement (RSI)

- **Core Idea: AI improves AI.**

  A system improves itself, keeps successful changes, and uses them to improve further.



- **Different Levels of RSI**

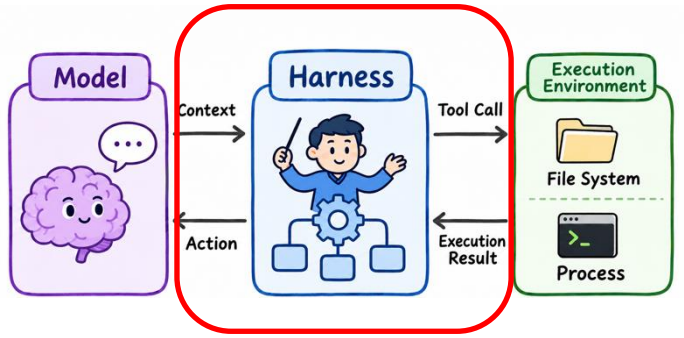


Harness RSI

➢ Why Harness RSI?

- **No model training required**
- Relatively **low-cost and practical**

} friendly to researchers



➢ Static RSI *VS* Open-World RSI

- **Static RSI**
  - Optimize on fixed tasks
  - Deploy the best configuration

Example: Fixed benchmark coding

The optimized harness is fixed.

- **Open-World RSI: Dynamic**

  Adapt to new tasks

Example: Deployed real-world coding agent (ongoing)

The harness evolves over time.

Example of Harness RSI

1. Find the issue

   The agent did not reconcile information across files or verify its final outputs.

2. Modify the harness **What to improve**

   Add reconciliation and verification to this harness.

3. Test the new harness

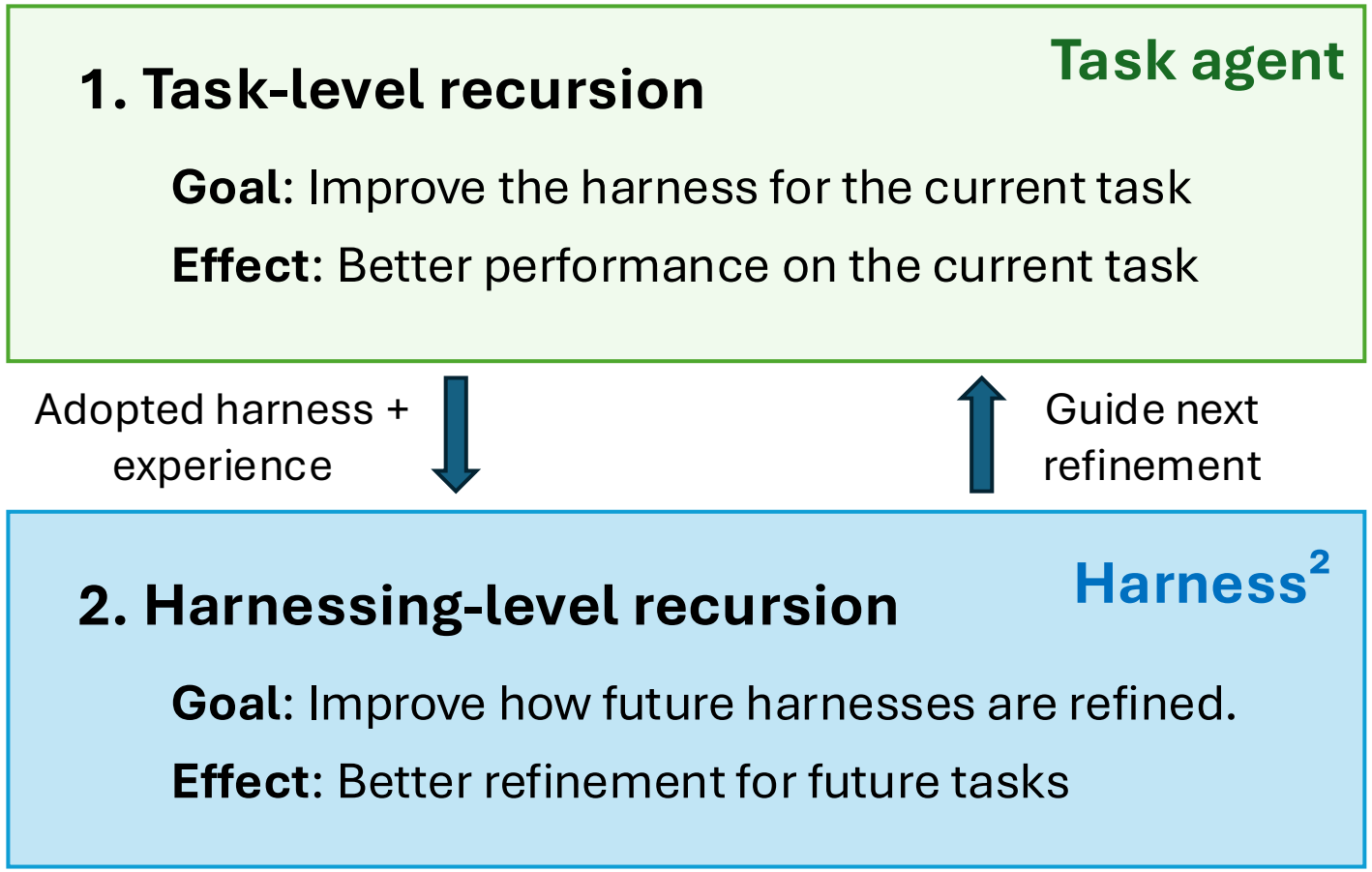
   43.1 → 77.8

4. Learn from this process **How to improve**

   When an agent fails on a multi-source task, first check whether the failure comes from missing reconciliation, provenance tracking, or output verification.

Method

Framework: Recursive Agent Harnessing



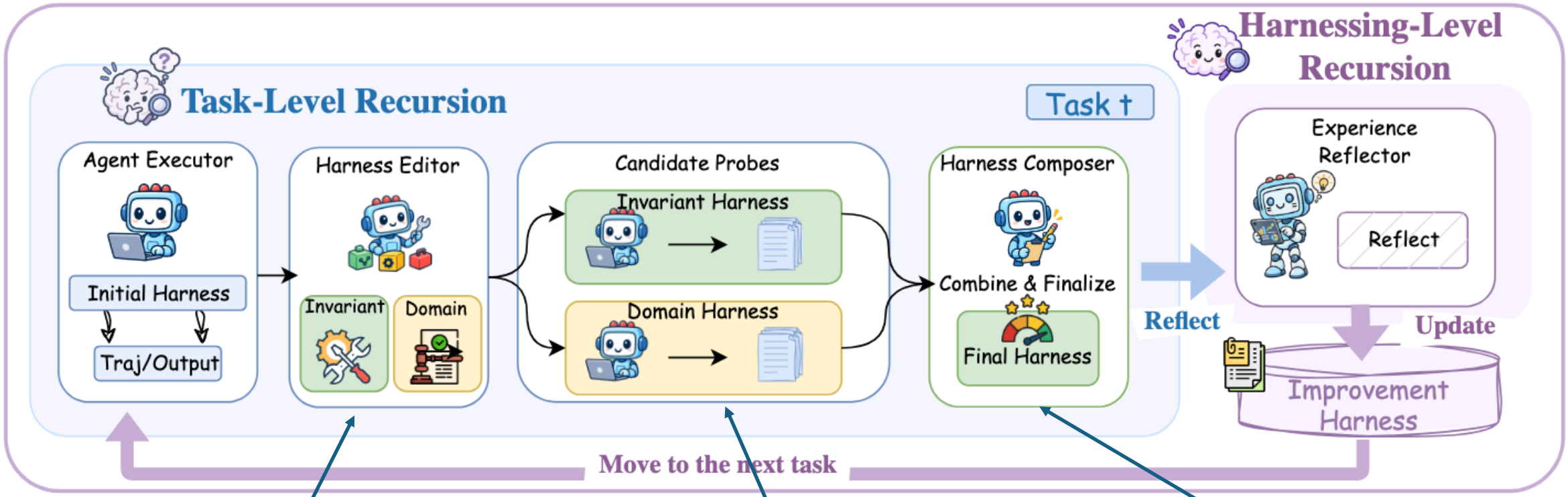
➢ **Harness² do two things:**

1. Improve the Task agent

2. Improving the harness that improves the harness.

(Why called Harness²)

1. Task-Level Recursion



Propose the Harness
Invariant: general fixes
Domain: task-specific fixes

What should we change?

Probe Candidates
Test and compare their effects.

What actually changed?

Compose Final Harness
Merge useful changes into the next harness.

Which changes should we keep?

1.1 Propose the Harness

1. Run the Current Harness

Task + Current Harness

Trajectory: reasoning, tool calls, etc.

Deliverable: the final output

2. Find an observed problem in the trajectory using LLM

Example: Analyze multiple Excel files

- ✓ Step 1: Read `employees.xlsx`
- ✓ Step 2: Read `departures.xlsx`
- ⚠ Step 3: Join records without checking unmatched IDs
- ✓ Step 4: Generate `report.xlsx`
- ⚠ Step 5: Finish without reopening the output

3. LLM Agent Acts as the Harness Editor

Prompt for editing harness

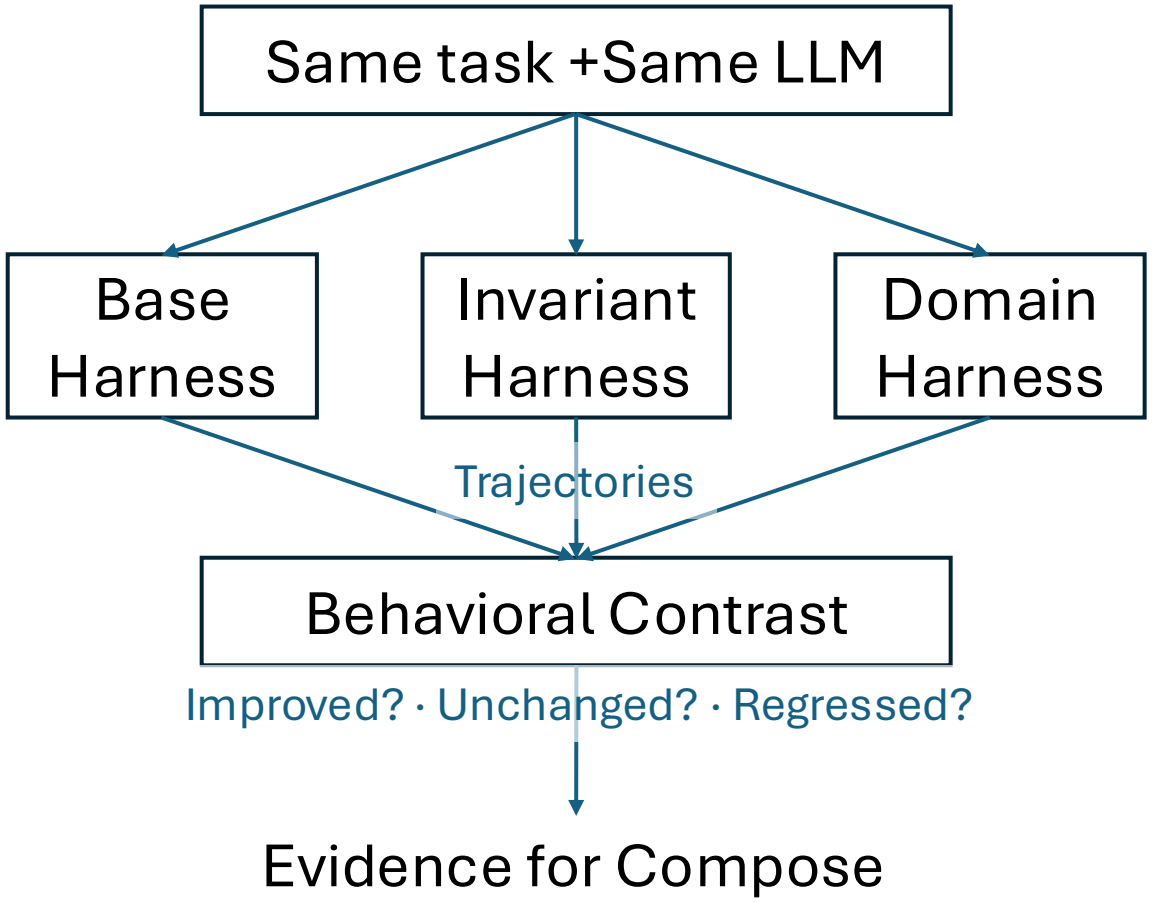
> Role: You are an INVARIANT Harness Editor.
>
> Available evidence:
> - Current task
> - Current harness files
>
> Instructions:
> 1. Inspect the execution trajectory.
> 2. Identify a concrete procedural weakness.
>
> Constraints:
> - Do not change what already works.
> - Keep the original harness if evidence is insufficient.

Two types: Invariant/Domain edit

Original Prompt:
https://github.com/google-research/harness2/blob/main/harness2/core/prompts.py

1.2 Candidate Probes

➢ Pipeline



- Base Trajectory

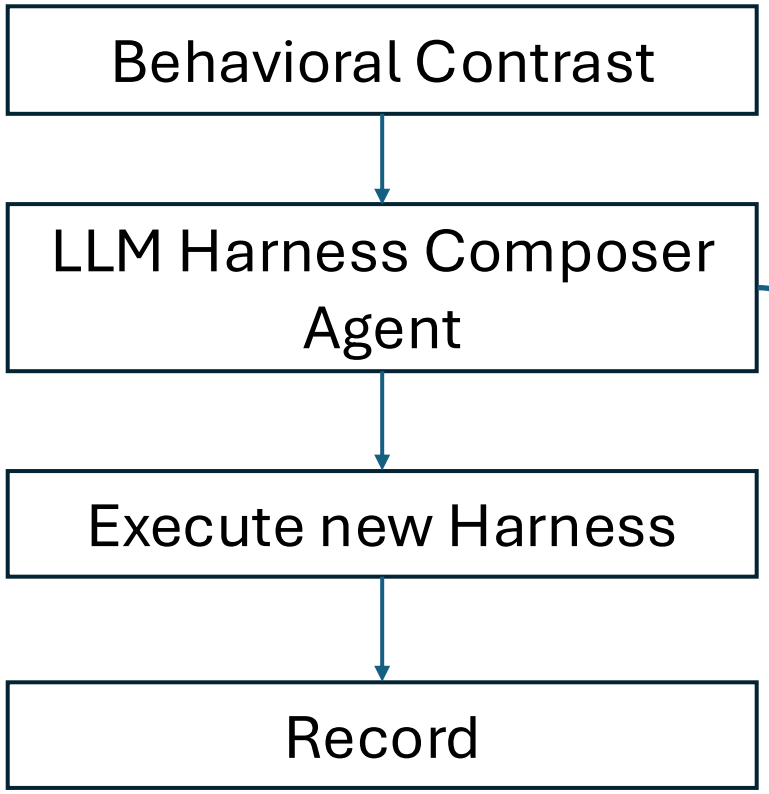
  - Step 1: Read `employees.xlsx`
  - Step 2: Read `departures.xlsx`
  - Step 3: Join records without checking unmatched IDs
  - Step 4: Generate `report.xlsx`
  - Step 5: Finish without reopening the output

➢ Behavioral Contrast

Observed behavior	Base	Invariant	Domain
Read source files	✓	✓	✓
Check unmatched IDs	✗	✗	✓
Reopen output	✗	✓	✗
Produce report	✓	✓	✓

❖ Probe provides behavioral evidence, not ground-truth correctness.

1.3 Harness Composer



Observed behavior	Base	Invariant	Domain
Read source files	✓	✓	✓
Check unmatched IDs	✗	✗	✓
Reopen output	✗	✓	✗
Produce report	✓	✓	✓

Select and Integrate Edits

Merge supported changes;
Revise conflicting edits;
Adopt one candidate;
Keep the base.

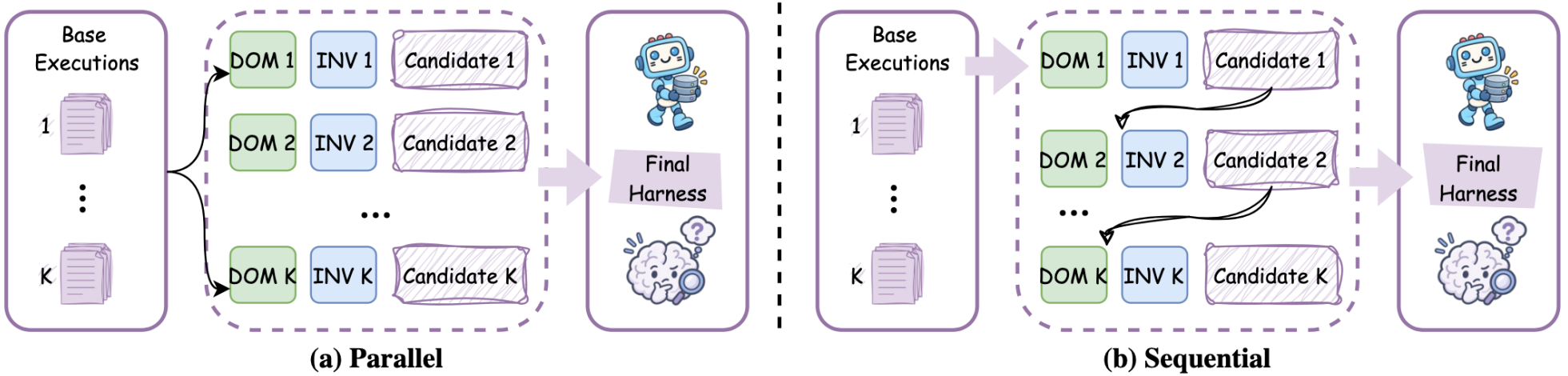
Multiple Candidate Harnesses

- Harness RSI Pipeline

Propose the Harness → **Probe Candidates** → **Compose Final Harness**

What should we change? What actually changed? Which changes should we keep?

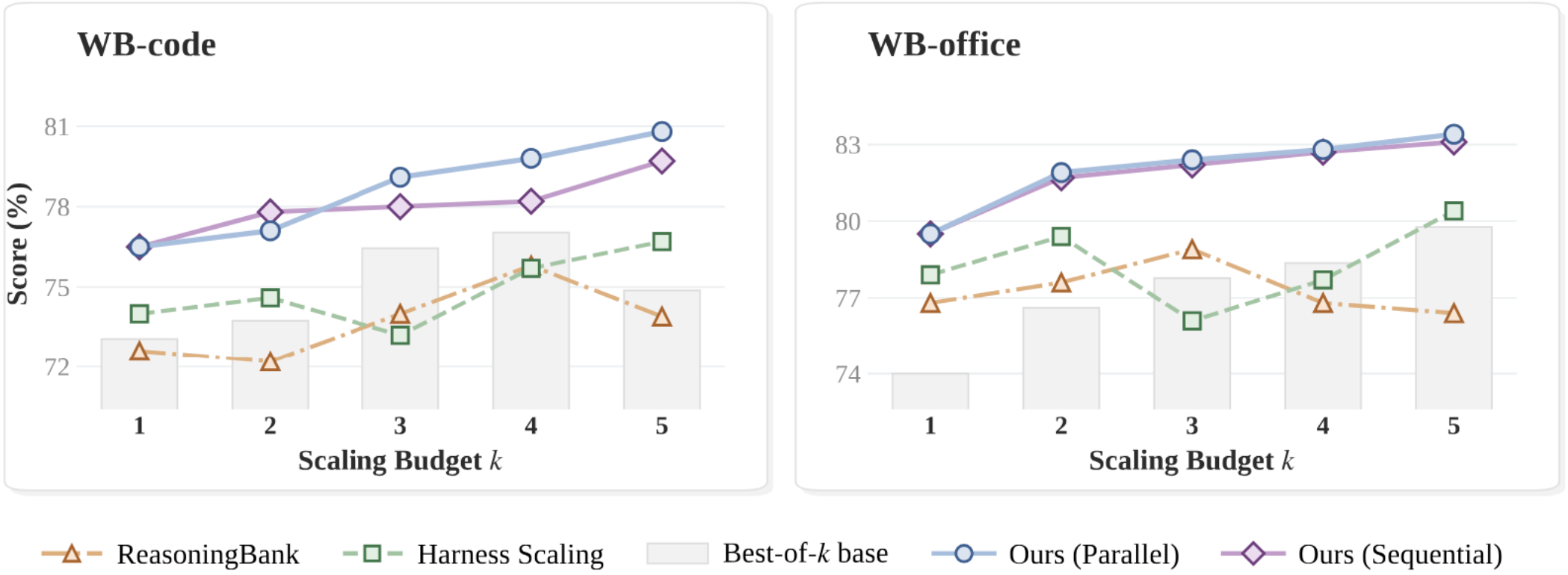
- Explore Diverse Improvements



Harness modifications are uncertain, and different edits may address different weaknesses.

Multiple Candidate Harnesses

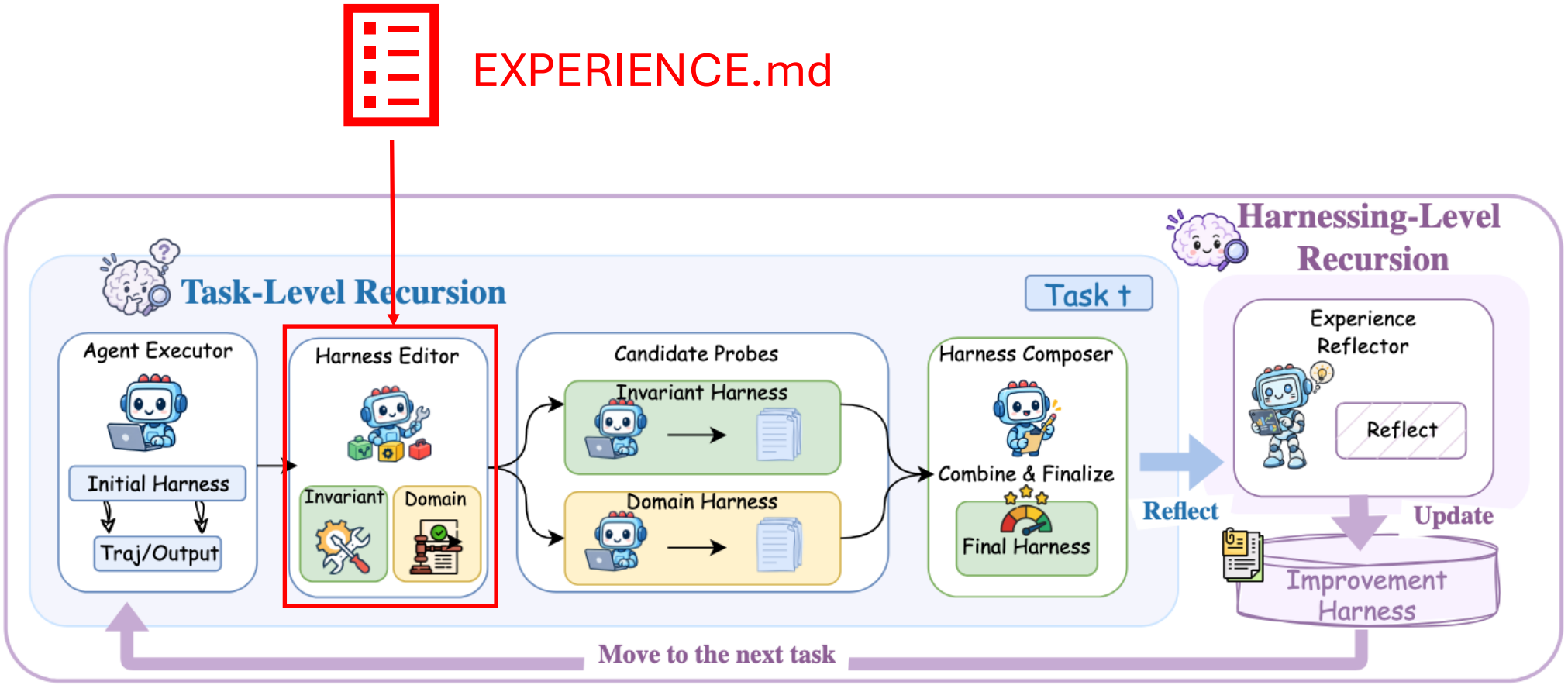
- Effect of Multiple Candidate Harness



k controls the exploration budget: k candidates

2. Harnessing-Level Recursion

- **Task-Level**：Improving the Agent Harness
- **Harnessing-Level**：Improving the Harness Improvement Process



Construct EXPERIENCE.md

1. Reflect on Previous Improvements

LLM Identify what worked, why, and under which conditions.

2. Update the Improvement Library

Record reusable experience in EXPERIENCE.md.

3. Reuse EXPERIENCE.md on Future Tasks

Data Structure in EXPERIENCE.md

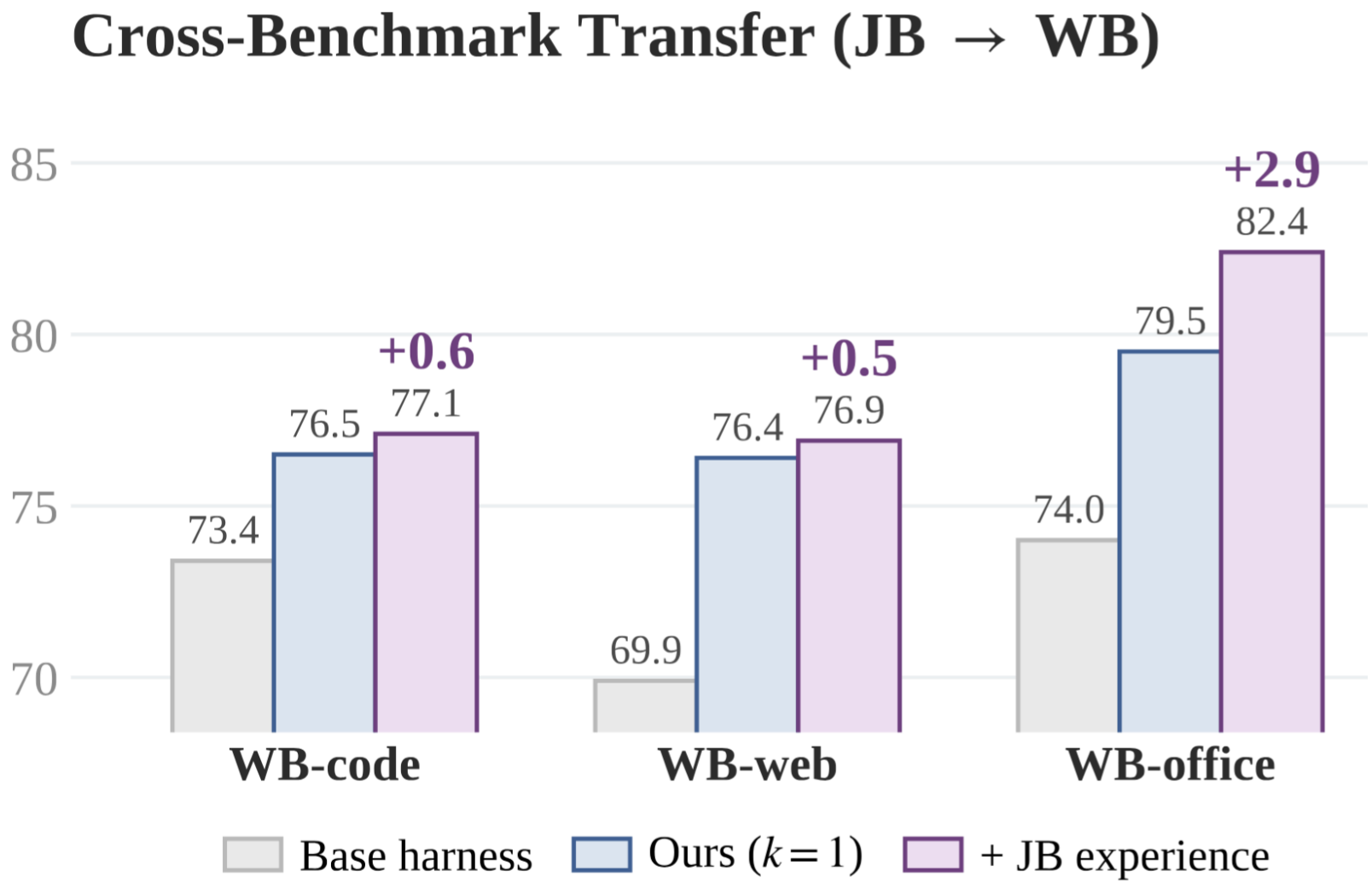
cue: When a task requires merging multiple datasets

lesson: Add a data-reconciliation skill to check unmatched records before aggregation

evidence: Candidate A performed the reconciliation step; the base execution omitted it

confidence: medium — supported by a limited number of executions

Effect of EXPERIENCE.md



❖ Experience learned from JB tasks also benefits WB tasks.

Experiments

Experiments

➢ Benchmarks



32 M users in China

WorkBuddy-Bench (Tencent): Real-world agent tasks, Office domains.

JobBench (Multi-institution Research Team): Long-horizon professional tasks.

➢ Implementation Details

- **Agent Harnesses:** OpenCode and Codex.





- **Executor LLMs:** Gemini 3.5 Flash and GLM-5 (frozen).
- **Harness Editor:** Claude Opus 4.8, running under OpenCode.

Results

- Main Results

Method	Code		Web		Office	
	Score	#Steps	Score	#Steps	Score	#Steps
OpenCode	73.4 (3.6)	29.6 (1.0)	69.9 (4.2)	19.2 (0.8)	74.0 (2.2)	27.3 (3.3)
HARNESS^2	**76.5** (1.3)	**22.8** (1.3)	**76.4** (4.5)	17.7 (0.7)	**79.5** (3.1)	25.1 (1.7)
+ Recursion	+4.3	−0.4	+1.9	+0.4	+3.9	−2.2
Codex	74.0 (1.7)	31.8 (0.9)	72.5 (1.2)	22.6 (0.9)	77.0 (2.2)	30.6 (1.2)
HARNESS^2	**79.4** (3.6)	**26.6** (2.8)	**74.7** (3.8)	**19.5** (1.1)	**78.5** (2.4)	**25.6** (1.3)
+ Recursion	+1.6	−2.0	+2.6	0.0	+4.3	+1.3

- **Harness^2** outperforms the original OpenCode and Codex harnesses.
- **+ Recursion** further improves performance by exploring multiple candidate harnesses.
- **Codex** generally outperforms OpenCode.

Thank you!

Happy to take questions.